INSTITUTO FEDERAL GOIANO — CAMPUS IPORÁ
Bacharelado em Ciência da Computação • 3º Período — 2026/2

Teoria dos Grafos — Lista de Exercícios 6: Árvores e Arborescências (Parte 2)

Docente: Prof. Newarney T. Costa | **Integrantes da Dupla:** Messias Junio da Silva Mendes & Bruno Alves Mendonça De Abreu Filho
Adaptação Acadêmica: Resolução integralmente digitada em conformidade com as diretrizes institucionais de acessibilidade funcional.

Questão 1 — Algoritmos de Kruskal e Prim: Conceitos, Complexidade e Estruturas

- **a) Estratégia de escolha das arestas:** O algoritmo de **Kruskal** ordena todas as arestas do grafo em ordem crescente de peso e escolhe, a cada passo, a aresta de menor peso global que não forme ciclo com as arestas já selecionadas. Já o algoritmo de **Prim** parte de um vértice inicial e expande uma única árvore conexa, escolhendo a cada passo a aresta de menor peso que conecta um vértice já pertencente à árvore a um vértice ainda não incluído.
- **b) Grafos não conexos:** Não necessariamente produzem uma única árvore. Em grafos desconexos, ambos produzem uma **floresta geradora mínima** (no caso do Kruskal, de forma direta; no caso do Prim, gerando a AGM da componente inicial e precisando ser reiniciado nas demais componentes conexas), em vez de uma única Árvore Geradora Mínima.
- **c) Complexidade e vantagem de cada abordagem:**
  - **Kruskal:** Complexidade **O(E log E)** (ou O(E log V)). É mais vantajoso em **grafos esparsos** (com poucas arestas, E ≈ V), pois a ordenação das arestas é rápida e simples.
  - **Prim:** Complexidade **O(E log V)** usando lista de adjacência e fila de prioridade (eficiente em grafos esparsos); em **grafos densos** (E ≈ V^2), a implementação com matriz de adjacência e vetor de custos mínimos atinge **O(V^2)**, sendo mais vantajosa por evitar o fator logarítmico sobre O(V^2) arestas.
- **d) Estruturas de dados utilizadas:**
  - **Kruskal:** Utiliza a estrutura **Union-Find** (Conjuntos Disjuntos) para verificar eficientemente se dois vértices pertencem à mesma componente conexa, evitando a formação de ciclos.
  - **Prim:** Utiliza uma **Fila de Prioridade (Min-Heap)** (ou vetor de custos mínimos *key[]*) para selecionar rapidamente a aresta de menor peso que conecta a árvore em construção a um vértice externo.

Questão 2 — Aplicação de Kruskal e Prim (Grafo A, B, C, D)

a) Aplicando o Algoritmo de Kruskal: Ordenando as arestas em ordem crescente de peso:

Ordem	Aresta	Peso	Decisão	Justificativa
1º	**A — B**	1	**Aceita**	Une os vértices A e B (não forma ciclo)
2º	**B — C**	2	**Aceita**	Une C ao componente {A, B} (não forma ciclo)
3º	**A — C**	3	Rejeitada	A e C já estão conectados (formaria o ciclo A–B–C–A)
4º	**B — D**	4	**Aceita**	Une D ao componente {A, B, C} (completa V - 1 = 3 arestas)
5º	**C — D**	5	Rejeitada	C e D já estão conectados (formaria o ciclo B–C–D–B)

AGM obtida por Kruskal: Arestas {A–B, B–C, B–D} | **Peso total:** 1 + 2 + 4 = **7**.

b) Aplicando o Algoritmo de Prim a partir do vértice A:

Passo	Vértices Incluídos na Árvore	Aresta Escolhida (Menor Peso no Corte)	Peso
1	{A, B}	**A — B**	1
2	{A, B, C}	**B — C**	2
3	{A, B, C, D}	**B — D**	4

AGM obtida por Prim: Arestas {A–B, B–C, B–D} | **Peso total:** 1 + 2 + 4 = **7**.

c) Comparação: Sim, neste grafo as árvores geradoras mínimas obtidas por Kruskal e Prim são idênticas: ambas contêm as arestas A–B, B–C e B–D, com peso total 7 (pois todos os pesos das arestas são distintos, garantindo unicidade da AGM). Isso nem sempre acontece: Kruskal e Prim podem produzir árvores estruturalmente diferentes quando existem múltiplas arestas com o mesmo peso em um ciclo ou corte; entretanto, ambos sempre encontram uma árvore geradora mínima com o mesmo custo total ótimo.

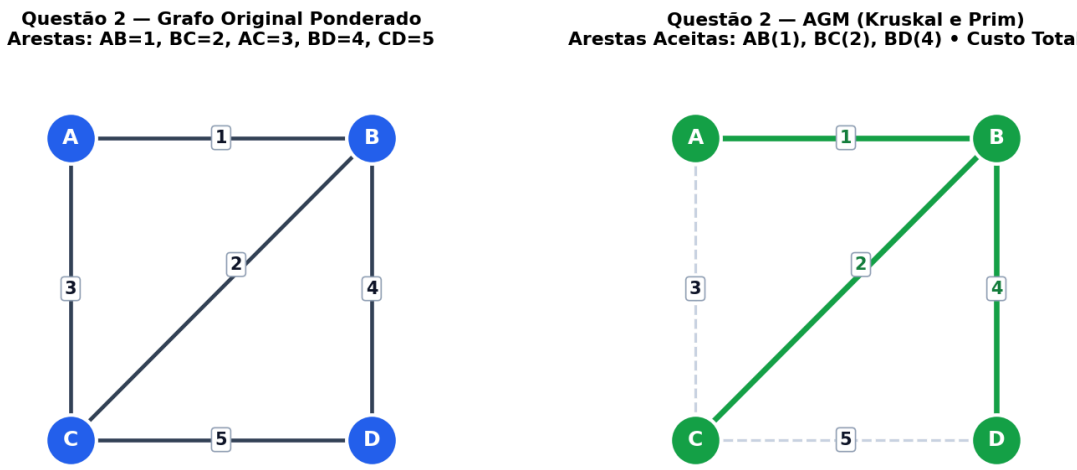


Figura 1 — Grafo original ponderado da Questão 2 e a Árvore Geradora Mínima resultante (custo total = 7).

Questão 3 — Algoritmo de Prim · Implementação em C e Python

3.1 Leia e Explique (Funcionamento Passo a Passo e Complexidade)

1. Explicação passo a passo do código em C:

- **Entrada e Matriz de Custos (linhas 7–15):** O programa lê a quantidade de vértices `n` e preenche a matriz `cost[n][n]` com os pesos das arestas entre cada par de vértices `(i, j)` (onde `0` indica ausência de aresta direta).
- **Vetor de Vértices Selecionados (linhas 17–19):** O vetor `selected[n]` controla quais vértices já fazem parte da Árvore Geradora Mínima (`1` = incluído, `0` = fora da árvore). Inicialmente todos recebem `0`, e o vértice `0` é marcado como raiz inicial (`selected[0] = 1`).
- **Laço Principal de Construção da AGM (linhas 21–22):** A variável `edges` conta quantas arestas já foram adicionadas à AGM e `mincost` acumula o peso total. O laço `while (edges < n - 1)` repete exatamente `n - 1` vezes, pois toda árvore com `n` vértices possui `n - 1` arestas.
- **Busca da Aresta de Menor Custo no Corte (linhas 23–35):** Em cada iteração do `while`, inicializa-se `min = INF`. O laço `for (int i = 0; i < n; i++)` filtra apenas os vértices `i` já pertencentes à árvore (`if (selected[i])`). Para cada um deles, o laço aninhado `for (int j = 0; j < n; j++)` examina todos os vértices `j` que ainda estão fora da árvore (`!selected[j]`) e possuem aresta com `i` (`cost[i][j] != 0`). Sempre que encontra um peso menor que `min`, atualiza `min = cost[i][j]` e registra os extremos `x = i` e `y = j`.
- **Atualização da Árvore (linhas 36–40):** Ao final da varredura dupla, imprime a aresta escolhida `(x, y)` e seu peso `min`, soma `min` a `mincost`, inclui o vértice `y` na árvore (`selected[y] = 1`) e incrementa `edges++`. Por fim (linha 41), imprime o custo mínimo total.

2. Identificação da Complexidade (considerando os 3 laços aninhados):

- O laço externo `while (edges < n - 1)` executa **n - 1 vezes** — ordem **O(n)**.
- O laço intermediário `for (int i = 0; i < n; i++)` percorre os `n` vértices; na k-ésima iteração do `while`, a condição `if (selected[i])` é verdadeira para exatamente `k` vértices.
- O laço mais interno `for (int j = 0; j < n; j++)` executa **n vezes** para cada vértice `i` selecionado.
- Somando sobre todas as iterações: $\sum_{k=1}^{n-1} (k \cdot n) = n \cdot n(n - 1)/2 = (n^3 - n^2)/2$. Portanto, a complexidade de tempo desta implementação é **$O(n^3)$** (ou **$O(V^3)$**).

3. Versão Eficiente vs. Versão Ingênua e Como Reduzir a Complexidade:

- Esta implementação corresponde à **versão mais simples/ingênua — $O(V^3)$**, e **não** às versões eficientes discutidas na Questão 1 (que são $O(V^2)$ com matriz ou $O(E \log V)$ com heap).
- **Justificativa:** Ela é ingênua porque, a cada nova aresta adicionada, descarta todo o trabalho anterior e varre novamente a matriz inteira testando todos os pares `(i, j)` entre vértices selecionados e não selecionados.
- **O que mudaria para reduzir a complexidade:**
  1. *Para reduzir para $O(V^2)$ mantendo a matriz de custos:* Basta criar um vetor auxiliar `key[n]` (que guarda o menor peso de aresta conhecido ligando cada vértice `j` fora da árvore a

algum vértice já na árvore) e um vetor `parent[n]`. Assim, em cada passo do `while`, usamos um único laço `for` de O(V) para achar o vértice y com menor `key[y]` e, após incluí-lo, outro laço `for` sequencial (não aninhado!) de O(V) apenas na linha `cost[y][j]` para atualizar `key[j] = min(key[j], cost[y][j])`. O custo por passo cai de $O(V^2)$ para O(V), resultando em **$O(V^2)$** no total.

2. *Para reduzir para O(E log V) em grafos esparsos:* Substituir a matriz por **listas de adjacência** e manter os vértices não visitados em uma **Fila de Prioridade (Min-Heap)** ordenada por `key[]`.

3.2 Complete (`// TRECHO FALTANTE`) e Consequência da Omissão

1. Linha que falta no código (Linha 78):

```
selected[y] = 1;
```

2. O que aconteceria se essa linha fosse omitida — o programa trava, ou termina e dá uma resposta errada? Por quê?

- **Comportamento:** O programa **NÃO trava** (não entra em loop infinito); ele **termina normalmente após n - 1 iterações, porém fornece uma resposta completamente errada**.
- **Por que não trava?** Porque a condição de parada do laço é `while (edges < n - 1)` e a linha `edges++;` (linha 79) continua sendo executada em toda iteração. Assim, `edges` cresce de `0` até `n - 1` e o laço encerra.
- **Por que dá uma resposta errada?** Porque, sem `selected[y] = 1;`, nenhum novo vértice é marcado como pertencente à árvore. O vetor `selected[]` permanece congelado com apenas `selected[0] = 1` e todos os demais iguais a `0`. Como resultado, em todas as `n - 1` repetições do laço `while`, os laços `for` examinam exclusivamente as arestas que saem do vértice `0` e **escolhem repetidamente a exata mesma aresta de menor peso incidente ao vértice 0** (por exemplo, no grafo das fábricas, escolhe 4 vezes seguidas a aresta `(0, 1)` de peso 2, resultando em custo 8 em vez de 16).

3.3 Implemente — Rede de Fibra Óptica entre as 5 Fábricas (C e Python)

Código Completo em Python (Google Colab):

```
INF = float('inf')
N = 5
cost = [
    [0, 2, INF, 6, INF],
    [2, 0, 3, 8, 5],
    [INF, 3, 0, INF, 7],
    [6, 8, INF, 0, 9],
    [INF, 5, 7, 9, 0],
]
selected = [False] * N
selected[0] = True

edges, mincost = 0, 0
while edges < N - 1:
    min_val = INF
    x, y = 0, 0
    for i in range(N):
```

```
            if selected[i]:
                for j in range(N):
                    if not selected[j] and cost[i][j] > 0:
                        if min_val > cost[i][j]:
                            min_val = cost[i][j]
                            x, y = i, j
        print(f"Aresta ({x}, {y}) incluida de custo {min_val}")
        mincost += min_val
        selected[y] = True
        edges += 1

    print(f"Custo minimo = {mincost}")
```

Código Completo do Bloco `while` em C (compilável com `gcc`):

```
        while (edges < n - 1) {
            int min = INF, x = 0, y = 0;
            for (int i = 0; i < n; i++) {
                if (selected[i]) {
                    for (int j = 0; j < n; j++) {
                        if (!selected[j] && cost[i][j]) {
                            if (min > cost[i][j]) {
                                min = cost[i][j];
                                x = i;
                                y = j;
                            }
                        }
                    }
                }
            }
            printf("Aresta (%d, %d) incluida de custo %d\n", x, y, min);
            mincost += min;
            selected[y] = 1;
            edges++;
        }
```

Saída Obtida (Idêntica em Python e C):

```
Aresta (0, 1) incluida de custo 2
Aresta (1, 2) incluida de custo 3
Aresta (1, 4) incluida de custo 5
Aresta (0, 3) incluida de custo 6
Custo minimo = 16
```

Teste de Mesa Passo a Passo (Rastreamento das 4 Iterações sobre as 5 Fábricas):

Iteração (edges)	Vértices Selecionados (selected == 1)	Arestas Candidatas no Corte (i ∈ S, j ∉ S)	Menor Aresta (x, y)	Custo (min)	mincost Acumulado
1 (edges = 0 → 1)	{0}	(0,1)=2, (0,3)=6	**(0, 1)**	2	2
2 (edges = 1 → 2)	{0, 1}	(1,2)=3, (1,4)=5, (0,3)=6, (1,3)=8	**(1, 2)**	3	5

Iteração (edges)	Vértices Selecionados (selected == 1)	Arestas Candidatas no Corte (i ∈ S, j ∉ S)	Menor Aresta (x, y)	Custo (min)	mincost Acumulado
3 (edges = 2 → 3)	{0, 1, 2}	(1,4)=5, (0,3)=6, (2,4)=7, (1,3)=8	**(1, 4)**	5	10
4 (edges = 3 → 4)	{0, 1, 2, 4}	(0,3)=6, (1,3)=8, (4,3)=9	**(0, 3)**	6	**16**

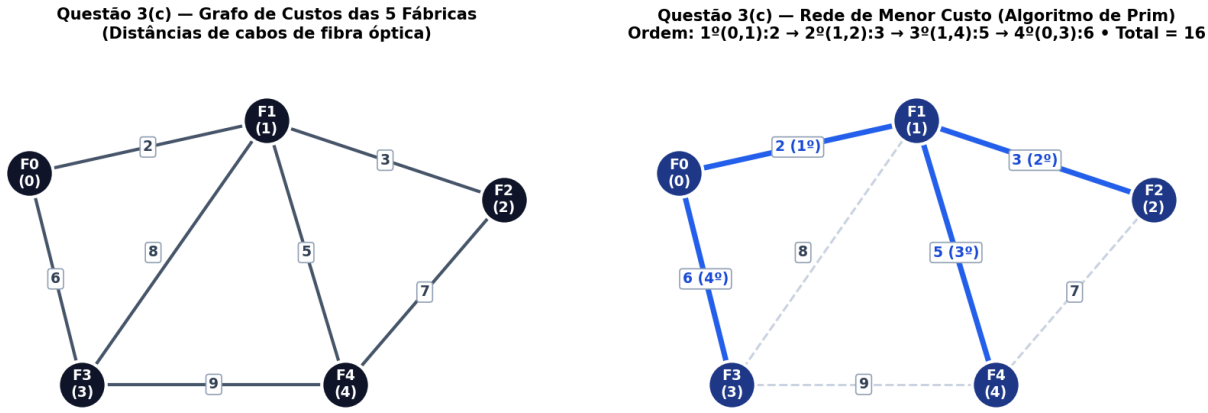


Figura 2 — Grafo de custos das 5 fábricas e a Árvore Geradora Mínima obtida pelo Algoritmo de Prim (custo mínimo = 16).